



Department of Electrical and Computer Engineering
North South University
CSE101 - Introduction to Python Programming

Planning Document

Travel Cost Planner and Option Comparator

Group Number: 04

Section: 2

Semester: Summer 2026

Ananto Shariar | 2211431042

Pranesh Majumder Tirtha | 2222899042

Md. Nasim Taif | 2212601042

Faculty Advisor

Silvia Ahmed (SvA)

Submission Deadline: 29 August 2026

2. Problem Statement

Planning a group trip requires several expenses to be considered together, including transportation, accommodation, food, activities, and other costs. When these expenses are calculated manually, mistakes can occur and it may be difficult to compare different plans or understand whether a trip fits the available budget.

The Travel Cost Planner and Option Comparator is intended for families, student groups, and small teams. The program will accept basic travel information, estimate the major cost categories, calculate the total cost and cost per person, check the budget, and compare alternative travel options. Its purpose is to make travel planning more organized, transparent, and easier to understand.

3. Scope

3.1 In Scope

- Create and store multiple travel options.
- Accept destination, number of travellers, transport, nights, room details, food, activities, other cost, and budget information.
- Calculate transport, accommodation, food, activity, other costs, subtotal, discount, final cost, and cost per person.
- Classify a plan as within budget, near budget, or over budget.
- Compare at least two travel options and explain the cost difference.
- Save and reload travel plans using a CSV file.
- Validate user input and handle common file problems safely.

3.2 Out of Scope

- Real-time airline, hotel, bus, or train prices.
- Online booking or payment processing.
- GPS, maps, or live location services.
- A graphical or web-based interface in the first version.
- Complex combinations of discounts or future price prediction.

The first version will remain a console-based Python program so that the group can focus on clear logic, reliable calculations, file handling, and testing.

4. Primary Functionality Requirements

4.1 Create Travel Option

The user will enter destination, number of travellers, transport details, number of nights, room information, food and activity costs, other costs, budget, comfort rating, and estimated travel time. Invalid values will be rejected before calculation.

4.2 Calculate Option Cost

The program will calculate transport, accommodation, food, activity, and other costs separately. It will then calculate the subtotal, discount, final estimated cost, and cost per person.

4.3 Apply Group Discount

The proposed rule is a 10% discount for groups of 5 or more travellers. Groups with fewer than 5 travellers will receive no discount. Only one discount will be used in the first version.

4.4 Budget Check

The program will show whether a travel option is Within Budget, Near Budget, or Over Budget. The proposed near-budget range is 90% to 100% of the available budget, and the program will also display the amount under or over budget.

4.5 Compare Options

At least two saved options will be compared using total cost, cost per person, budget status, comfort rating, and estimated travel time. The program will show the cheaper option, the cost difference, and a short recommendation.

4.6 Save and Reload Plans

Travel plans will be saved in a CSV file so that they can be loaded again in later program runs.

5. Inputs, Processing, and Outputs

5.1 Inputs

Option name: identifies a saved travel plan.

Destination: the place to be visited.

Travellers: the number of people travelling.

Transport type: bus, train, air, car, or another mode.

Fixed transport cost: any fixed transport charge.

Transport cost per person: the transport cost for one traveller.

Nights: the number of nights at the destination.

Room capacity: the number of people allowed in one room.

Room cost per night: the price of one room for one night.

Food cost per person per day: the estimated daily food expense.

Activity cost per person: the estimated activity expense.

Other cost: additional fixed travel expenses.

Budget: the maximum amount available for the trip.

Comfort rating: a rating from 1 to 5.

Travel time: the estimated journey duration.

5.2 Processing Rules and Formulas

Transport cost: $\text{Fixed Transport Cost} + (\text{Transport Cost per Person} \times \text{Travellers})$

Rooms required: $\text{Travellers} \div \text{Room Capacity}$, rounded upward when necessary.

Accommodation cost: $\text{Rooms Required} \times \text{Room Cost per Night} \times \text{Nights}$

Travel days: $\text{Nights} + 1$

Food cost: Food Cost per Person per Day x Travellers x Travel Days

Activity cost: Activity Cost per Person x Travellers

Subtotal: Transport + Accommodation + Food + Activity + Other Cost

Discount: 10% of the subtotal when Travellers ≥ 5 ; otherwise 0.

Final cost: Subtotal - Discount

Cost per person: Final Cost / Travellers

For example, if 7 travellers share rooms with a capacity of 3, the program will require 3 rooms because the result must be rounded upward.

5.3 Outputs

- Category-wise cost breakdown.
- Subtotal and discount amount.
- Final estimated travel cost.
- Cost per person.
- Budget status and amount under or over budget.
- Comparison of selected options.
- Recommended option with a short reason.

6. Planned Data Structures

The proposed design uses simple Python data structures that match the information being stored and are easy to explain during the demonstration.

6.1 List

A list will store multiple travel-option dictionaries. This allows the program to loop through saved plans, display them, and compare alternatives.

```
travel_options = [option1, option2]
```

6.2 Dictionary

A dictionary will represent one travel option using meaningful keys. This makes each value easy to identify and access.

```
option = {"destination": "Cox's Bazar", "travellers": 5, "budget": 30000}
```

6.3 Cost Dictionary

A second dictionary may store calculated cost categories so that the cost breakdown remains organized.

```
costs = {"transport": 6000, "food": 6000, "activities": 2500}
```

6.4 Tuple

A tuple may be used for fixed information, such as a selected travel date range, because those values are not expected to change during a calculation.

```
trip_dates = ("05-09-2026", "07-09-2026")
```

7. Program Decomposition

The program will be divided into small functions. The main menu and user interaction will remain in `main.py`, while reusable calculation functions will be placed in `travel_utils.py` and imported into the main program.

`calculate_transport_cost(fixed_cost, per_person, travellers)`: returns `transport_total`; calculates total transport expense.

`calculate_accommodation_cost(travellers, capacity, room_cost, nights)`: returns `rooms_required` and `accommodation_total`.

`calculate_food_cost(travellers, days, food_rate)`: returns `food_total`.

`calculate_activity_cost(travellers, activity_rate)`: returns `activity_total`.

`calculate_subtotal(costs)`: returns subtotal by adding the cost categories.

`apply_travel_discount(subtotal, travellers)`: returns discount and `final_cost`.

`calculate_cost_per_person(final_cost, travellers)`: returns the estimated cost for one traveller.

`check_budget(final_cost, budget)`: returns budget status and difference.

`compare_options(option_list)`: returns a comparison and recommendation.

`save_plan(option_data, filename)`: saves one travel option to the CSV file.

`load_plans(filename)`: returns a list of previously saved travel options.

The separate module `travel_utils.py` is suitable for reusable calculation and budget functions because it keeps processing logic separate from menu and input handling in `main.py`.

8. Flowcharts

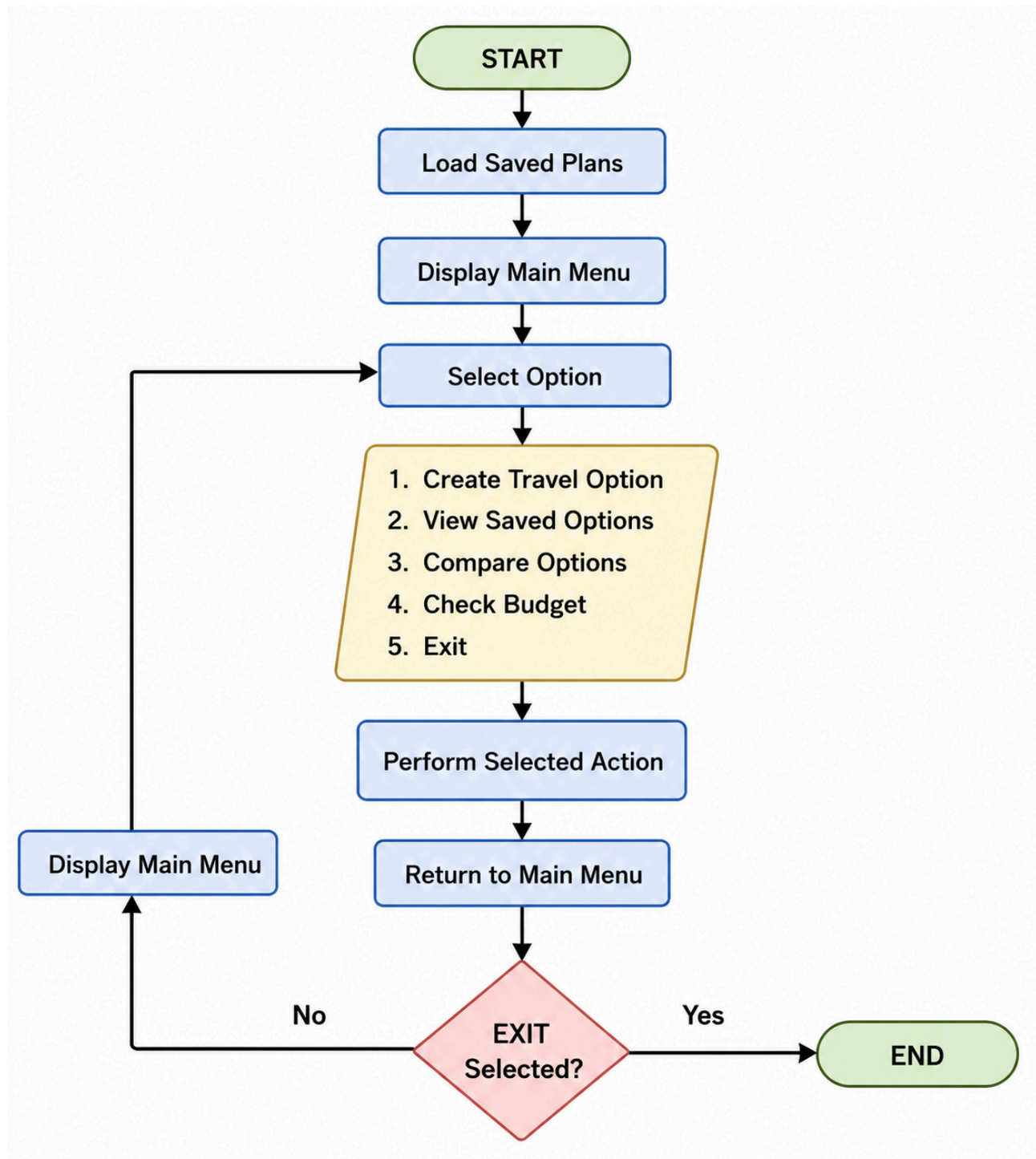


Figure 1. Proposed overall program flow.

9. File Design

The project will use a CSV file for persistent storage so that travel plans remain available between program runs.

9.1 File Name and Format

File name: travel_plans.csv

Proposed columns: option_name, destination, travellers, transport_type, transport_cost, nights, rooms_required, accommodation_cost, food_cost, activity_cost, other_cost, subtotal, discount, final_cost, cost_per_person, budget, budget_status, comfort_rating, travel_time.

9.2 Missing or Empty File

If travel_plans.csv is missing, the program will begin with an empty list and create the file when a plan is first saved. If the file exists but contains no records, the program will display "No saved travel plans found." and continue normally. Basic exception handling will be used so that invalid stored values do not cause the program to crash.

10. Validation and Test Plan

10.1 Validation Rules

- Travellers must be greater than 0.
- Nights must be 0 or greater.
- Room capacity must be greater than 0.
- Costs must not be negative.
- The budget must be greater than 0.
- Comfort rating must be between 1 and 5.
- Numeric fields must contain valid numbers.
- Menu choices must match the available options.

If an input is invalid, the user will receive a clear message and will be asked to enter the value again.

10.2 Planned Test Cases

Normal case: 4 travellers with valid costs -> Correct totals and budget status are displayed.

Normal comparison: Two saved options -> Options are compared successfully.

Boundary case: Travellers = 1 -> Program calculates normally.

Boundary case: Nights = 0 -> Accommodation cost becomes 0.

Room boundary: 7 travellers, room capacity = 3 -> Rooms required = 3.

Discount boundary: Travellers = 4 -> No discount is applied.

Discount boundary: Travellers = 5 -> 10% discount is applied.

Budget boundary: Final cost = 90% of budget -> Status is Near Budget.

Invalid input: Travellers = 0 -> Input is rejected.

Invalid input: Room capacity = 0 -> Input is rejected.

Invalid input: Budget entered as text -> Error is handled without crashing.

File error: CSV file is missing -> Program continues safely.

Empty file: No saved records -> A clear no-plans message is shown.

Comparison error: Only one option exists -> User is informed that at least two options are required.

Actual results and pass/fail status will be recorded in the final report after implementation and testing.

11. Work Distribution

The following work distribution is proposed for implementation. Each member will also review the integrated program and be prepared to explain the complete workflow during the demonstration.

Md. Nasim Taif | 2212601042: Main menu, travel-option input, input validation, transport-cost calculation, and main.py integration.

Pranesh Majumder Tirtha | 2222899042: Accommodation and room calculation, food and activity cost calculation, discount logic, and budget checking.

Ananto Shariar | 2211431042: Travel-option comparison, cost-per-person calculation, recommendation logic, CSV save/load, and file-error handling.

11.1 Integration Plan

Each member will first develop and test their assigned part. After that, all functions will be connected through main.py. Then all member will jointly test the complete program, including travel-cost calculation, budget checking, option comparison, file handling, and invalid-input cases.

12. Open Questions

The following decisions are still open to revision during implementation. The group will use one consistent rule and explain any significant changes in the final report.

Room calculation when group size is not divisible by room capacity. Proposed approach: round upward so that every traveller has accommodation.

Different child food or transport rates. Proposed approach: use the same rate for all travellers in Version 1; child pricing may be considered later.

Meaning of "near budget". Proposed approach: use 90% to 100% of the available budget as the warning range.

How comfort and travel time affect the recommendation. Proposed approach: first prefer options that fit the budget. If two suitable options are within about 5% in cost, use comfort as a tie-breaker and also display travel time.

How uncertain prices are communicated. Proposed approach: label the result as an Estimated Travel Cost and state that actual prices may vary.